Transition DS + Algorithms

BFS &
DFS

Worksheet 2

wolfchambers.github.io

Graphs

1. Data structures recap: For each of the following claims, explain what is missing or incorrect, or defend why the statement makes sense.

   (a) "Linked lists are best when there are many insertions, because insertion takes $O(1)$ time."

   (b) "Hash tables are always better than AVL trees, because $O(1)$ is better than $O(\log n)$."

   (c) "Arrays are poor choices for changing collections, because every insertion takes $O(n)$ time."

b) also: Rehashing Hash is worst case $O(n)$. AVL is steady worst case, + sorted

a) Only if have ptr; if no finding is $O(n)$

vectors are also $O(1)$ at end

Hashing is $O(1)$ expected also.

(c) not if at end!

2. Consider the adjacency matrix and adjacency list representations of a graph. Which choice would be better in each of the following situations? Justify your answer.

   (a) The graph has 10,000 vertices and 20,000 edges, and it is important to use as little space as possible. → adj list

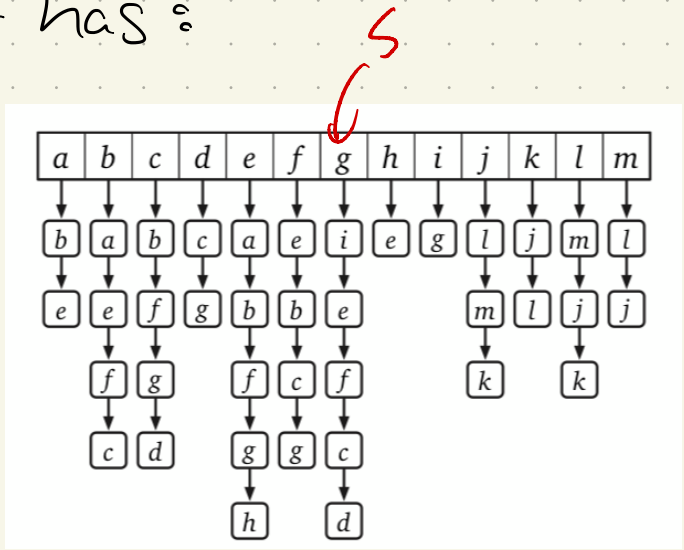
   (b) The graph has 10,000 vertices and 20,000,000 edges, and it is important to use as little space as possible.

   (c) You need to answer queries to find if two vertices are adjacent as fast as possible, no matter how much space you use. → adj matrix

Graph Searching

How can we tell if 2 vertices are connected?

Remember the computer only has:



Traverse lists & find who can reach who.



Bigger question: can we tell if all the vertices are in a single connected component?

Possibly you saw depth first search (DFS) and breadth first search (BFS) in data structures:

"bag": abstraction of both algorithm

```
WHATEVERFIRSTSEARCH(s):
  put s into the bag
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          for each edge vw
              put w into the bag
```

"source"

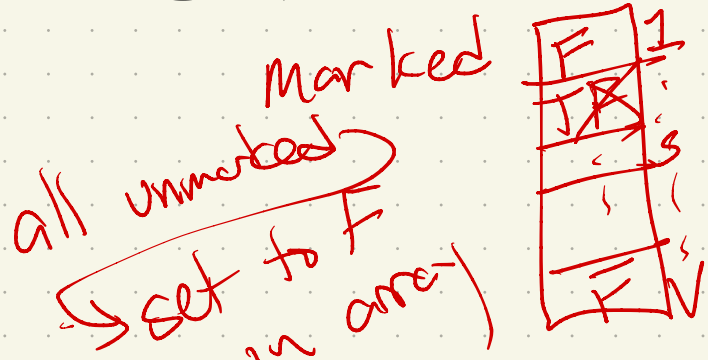
recursive vs iterative

in back

loop based

These are essentially just search strategies:

How can we decide if u ↔ v are connected?



bag: ∉ all s's nbrs

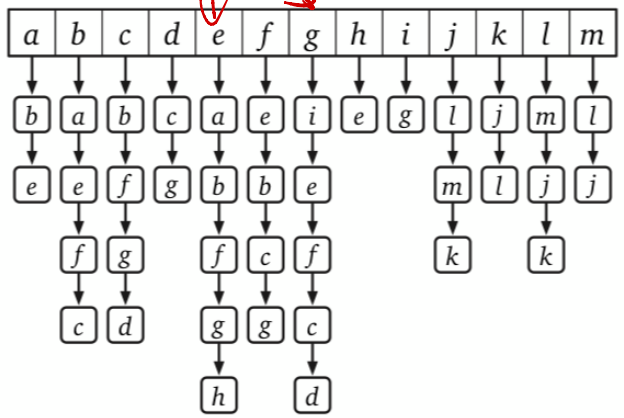
Can use this to build a spanning tree:

```
WHATEVERFIRSTSEARCH(s):
  put (∅, s) in bag
  while the bag is not empty
      take (p, v) from the bag          (⋆)
      if v is unmarked
          mark v
          parent(v) ← p
          for each edge vw              (†)
              put (v, w) into the bag   (⋆⋆)
```

tree

queue

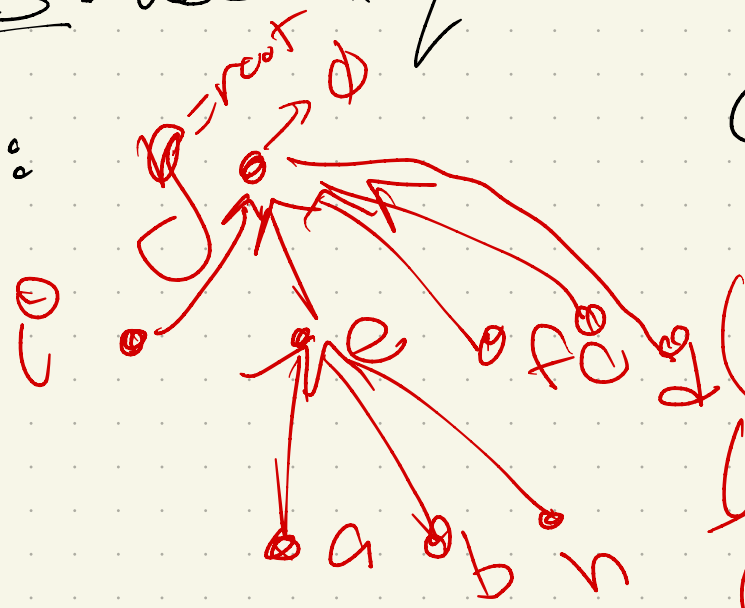
BFS(s)



marked
a
b
c
d
e
f
g
h
i
j
k
l
m

BFS: use a queue:

Tree:



Queue:

(s,i) (s,e) (s,f) (s,c) (s,d)
(e,a) (e,b) (e,f) (e,g)
(e,h) (f,e) (f,b) (f,c) (f,g)
c's nbr, d's nbrs, a's, b's

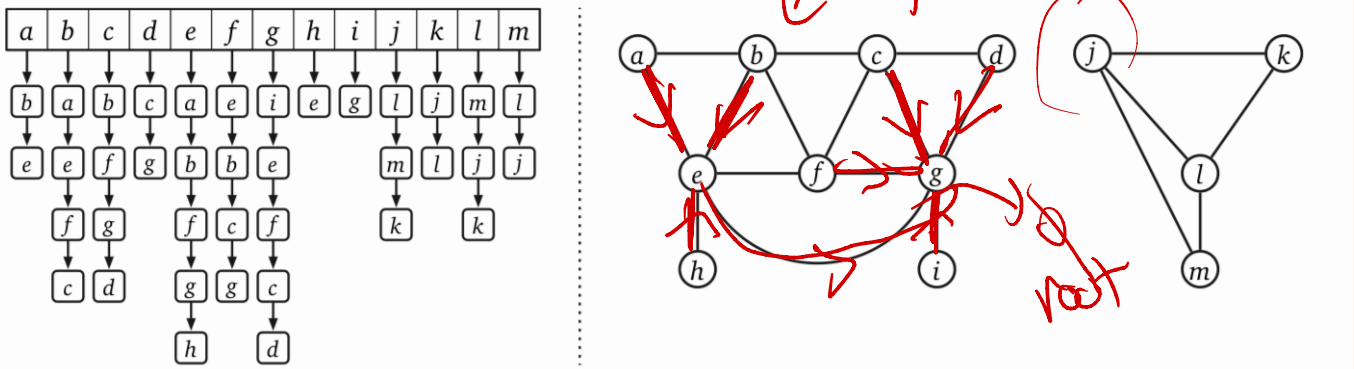
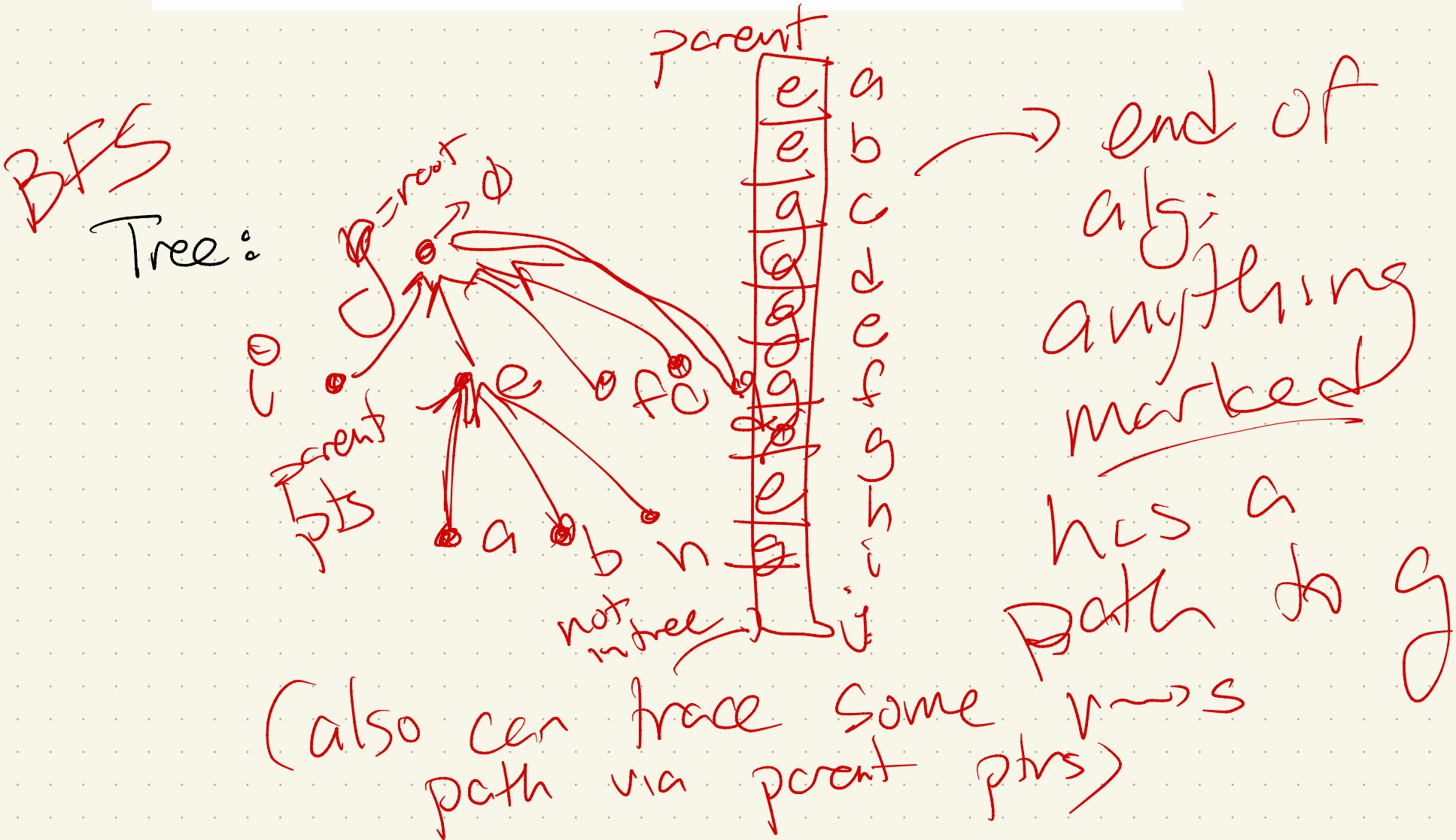
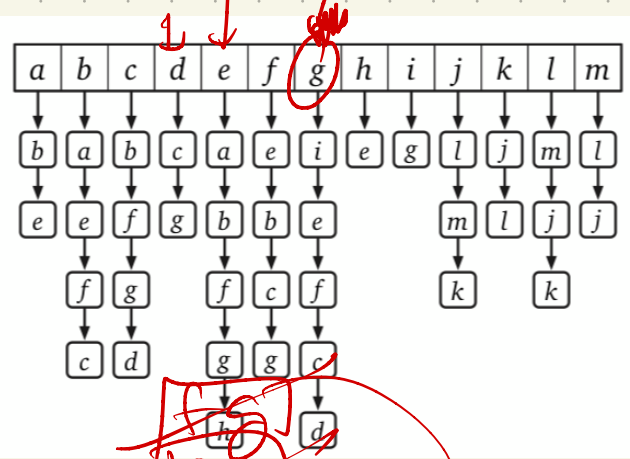


Figure 5.9. An adjacency list for our example graph.



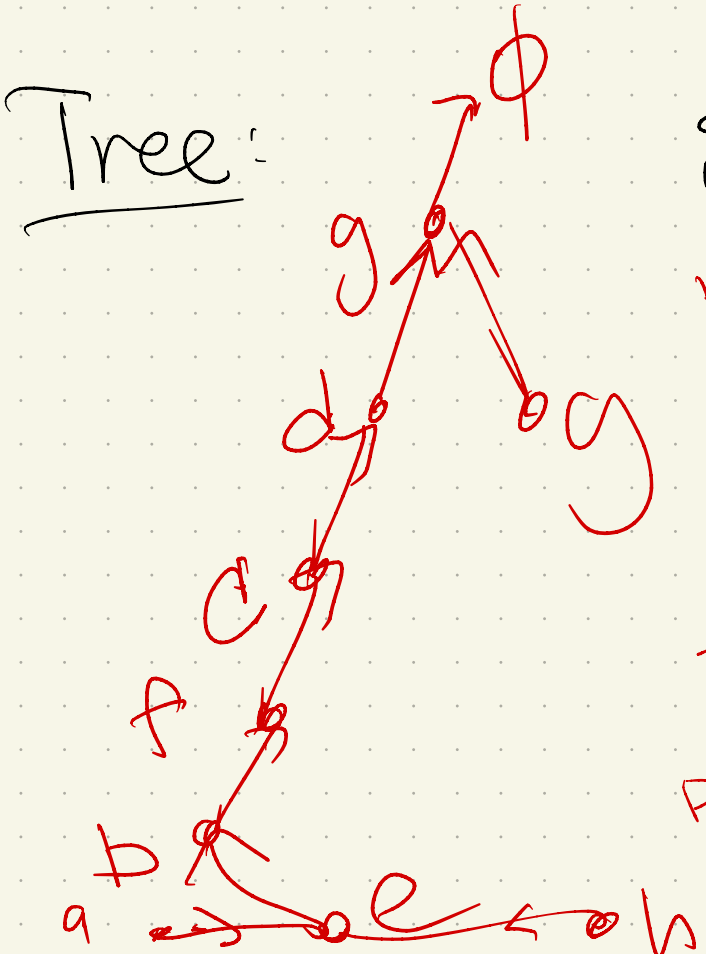
DFS: use a stack

```
WhateverFirstSearch(s):
  put (∅, s) in bag
  while the bag is not empty
      take (p, v) from the bag          (⋆)
      if v is unmarked
          mark v
          parent(v) ← p
          for each edge vw              (†)
              put (v, w) into the bag   (⋆⋆)
```

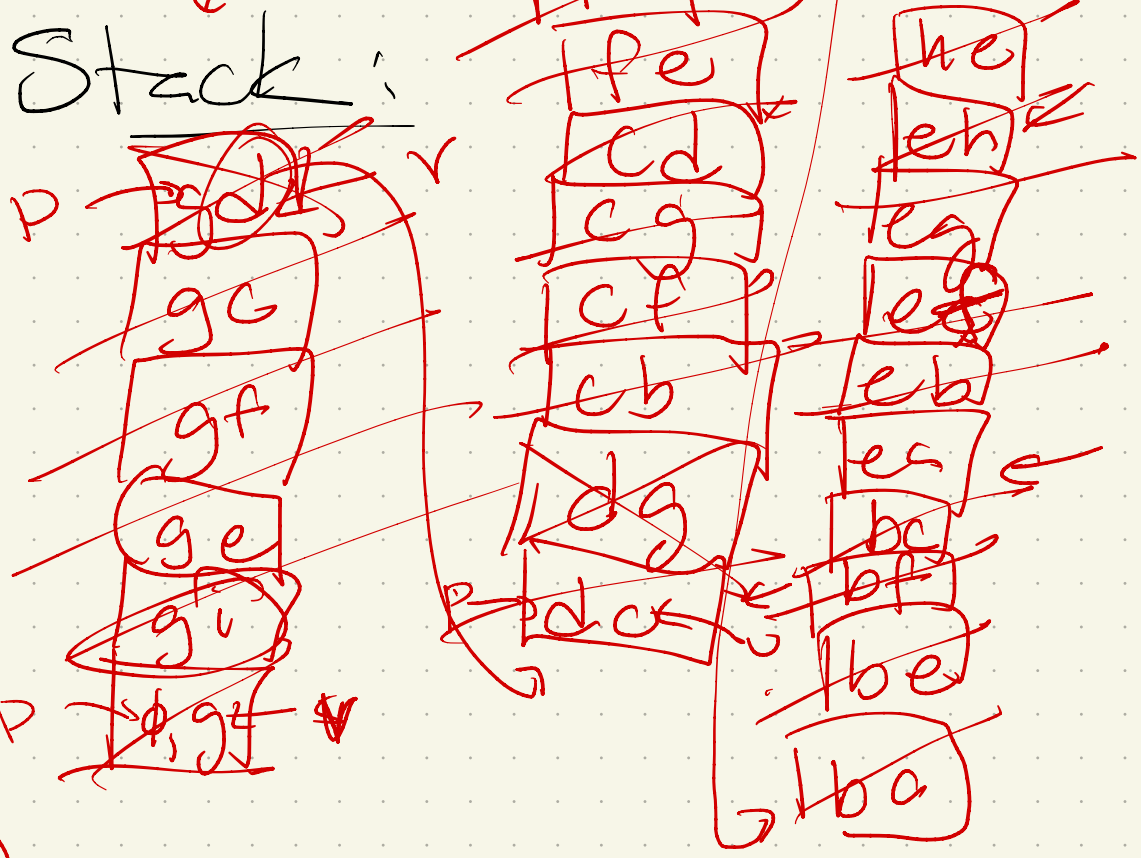


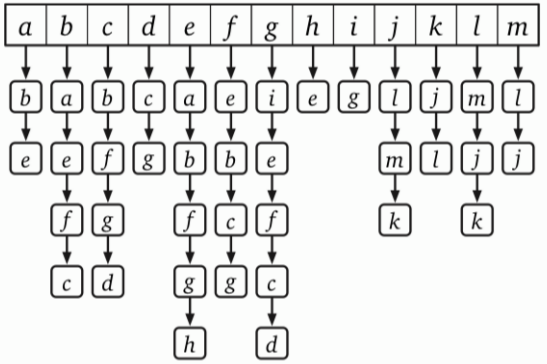
a b c d e f g h i j k l m

Tree:



Stack:





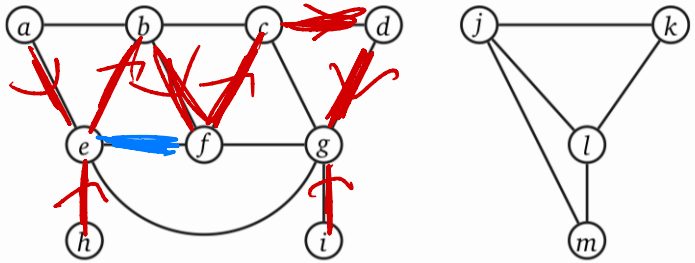
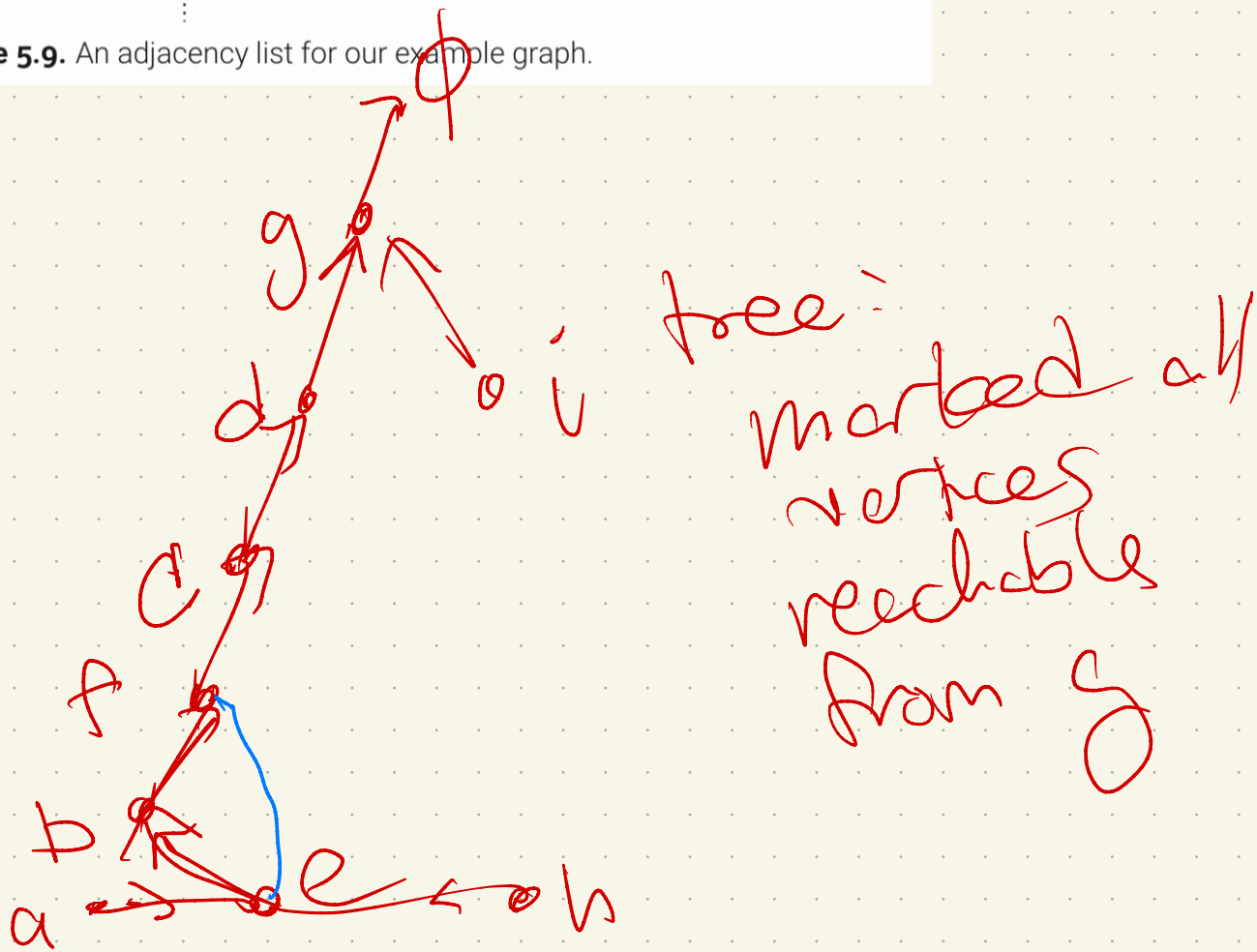


Figure 5.9. An adjacency list for our example graph.



Just remember: different!

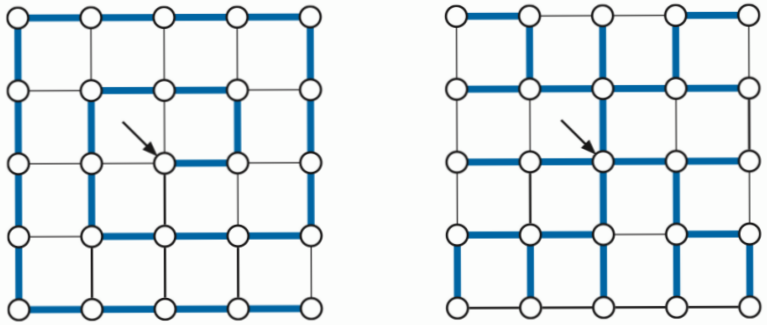
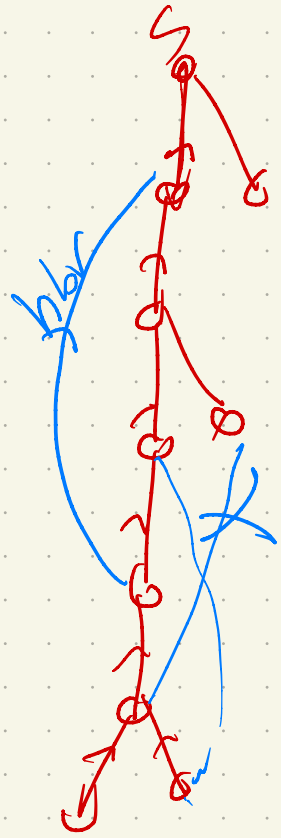


Figure 5.12. A depth-first spanning tree and a breadth-first spanning tree of the same graph, both starting at the center vertex.

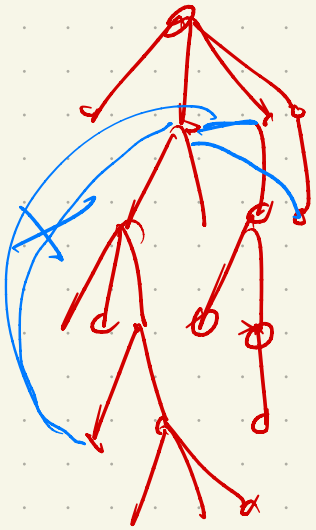


DFS:

All non-tree edges must connect a vertex to an ancestor in the tree

BFS:

All non-tree edges must connect vertices either at the same level, or 1 level apart



Correctness:

Claim: WFS will mark all reachable vertices.

pf: induction on distance to the source:

$d=0$:

$d>0$: Consider v at distance d in G_1 so

$$s \to v_1 \to v_2 \cdots \to v_{d-1} \to v_d$$

(d edges)

By IH:

Runtime:

```
WHATEVERFIRSTSEARCH(s):
  put s into the bag
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          for each edge vw
              put w into the bag
```

Claim: marked v's + parents form a spanning tree.

proof:

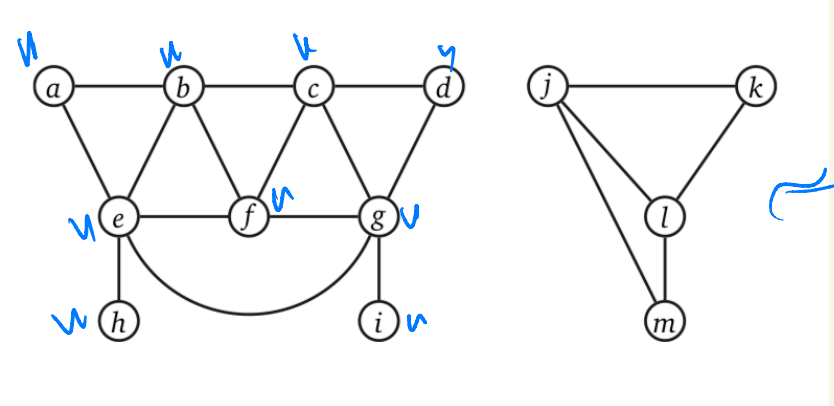
```
WHATEVERFIRSTSEARCH(s):
  put (∅, s) in bag
  while the bag is not empty
      take (p, v) from the bag                (⋆)
      if v is unmarked
          mark v
          parent(v) ← p
          for each edge vw                    (†)
              put (v, w) into the bag         (⋆⋆)
```

For each marked vertex:

In a disconnected graph:

Often want to count or label the components of the graph.

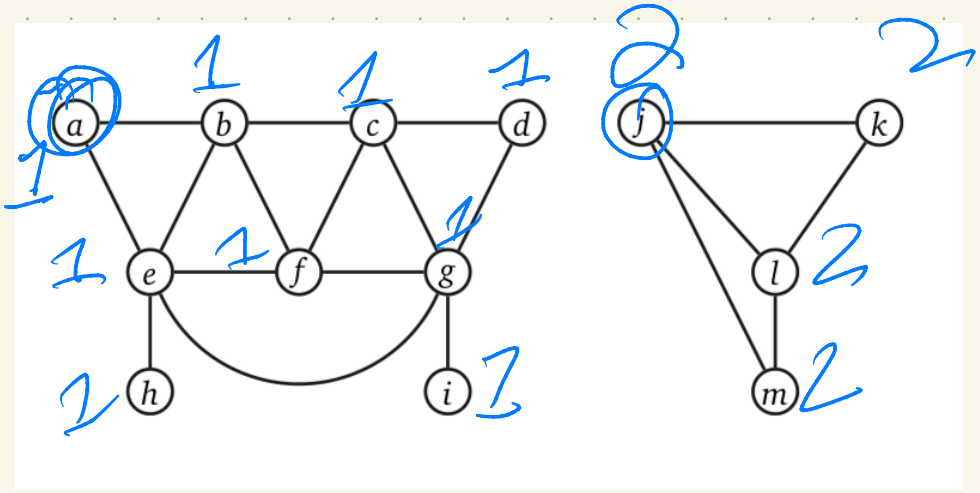
(WFS(v) will only visit the piece that v belongs to.)



Solution: while unmarked vertices remain, repeat the WFS

Modification: Might want to count the # of connected components:

```
COUNTCOMPONENTS(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          WHATEVERFIRSTSEARCH(v)
  return count
```

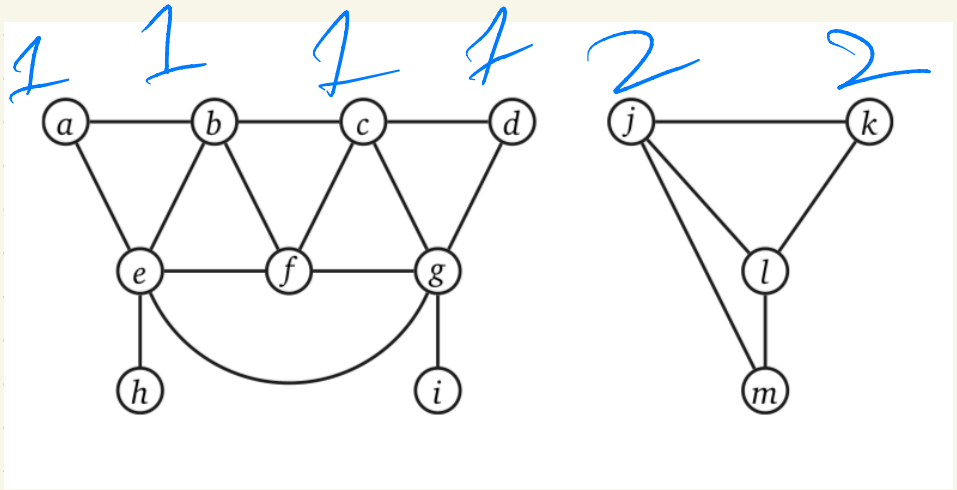


earlier code return 2

Finally, can even record which component each vertex belongs to:

```
CountAndLabel(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          LabelOne(v, count)
  return count
```

```
⟨⟨Label one component⟩⟩
LabelOne(v, count):
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          comp(v) ← count
          for each edge vw
              put w into the bag
```



Dfn: Reduction

A reduction is a method of solving a problem by transforming it to another problem.

Why? Reuse code!

conn comps

Lused WFS

Reduction graph example:

Given a pixel map, the flood-fill operation lets you select a pixel & change the color of it & all the pixels in its region.

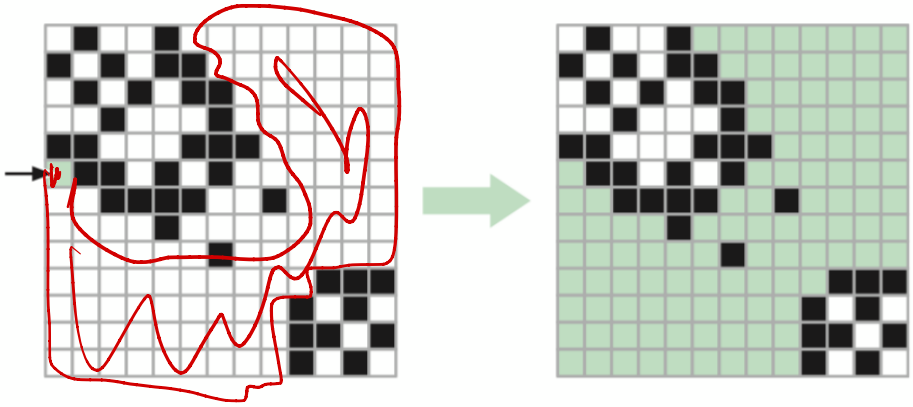
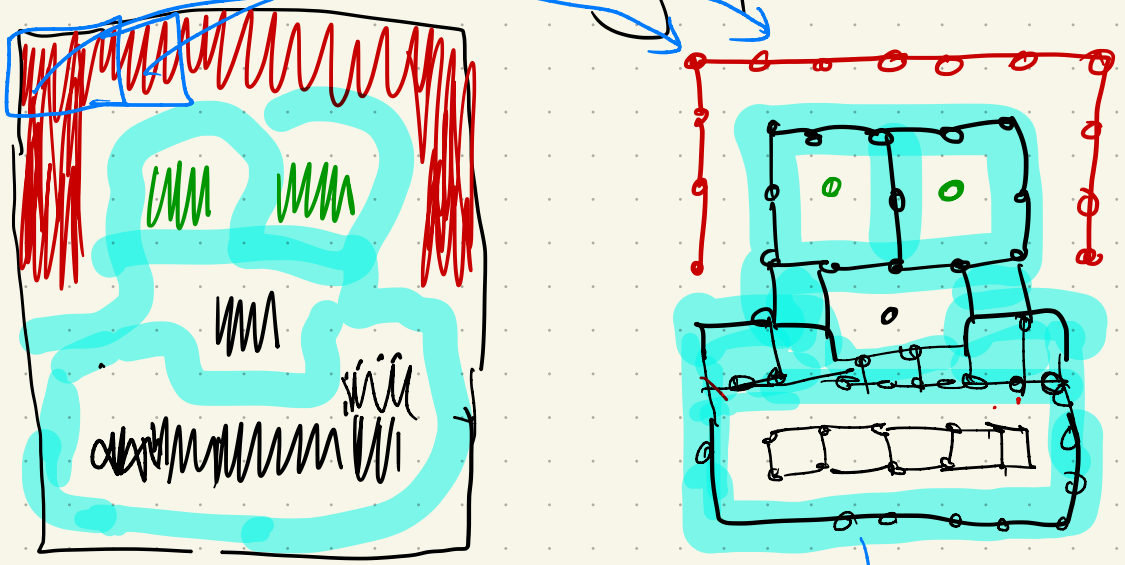


Figure 5.13. An example of flood fill

How to implement?

use BFS

So: Build a graph from pixels:



each pixel
↳ vertex in G

add edge b/t any two vertices whose pixels are in same region

n×n → graph with n^2 vertices

Algorithm

If pixel p is selected:

Find connected comp of p's equiv. vertex in graph

recolor anything in that comp

Arguably, these reductions are the most important thing in graphs!

Like data structures - you won't usually have to re-code everything.

Instead:

- Set up graph — Construct V & E
- Call some algorithm
  - ↳ WFS - (BFS or DFS)
    - connected comps
    - ⋮

Build graph:

let each of $n \times n$ spaces be a vertex

$V = \{(i,j) : 1 \le i,j \le n\}$

Create edges: for spot/vertex i,j,

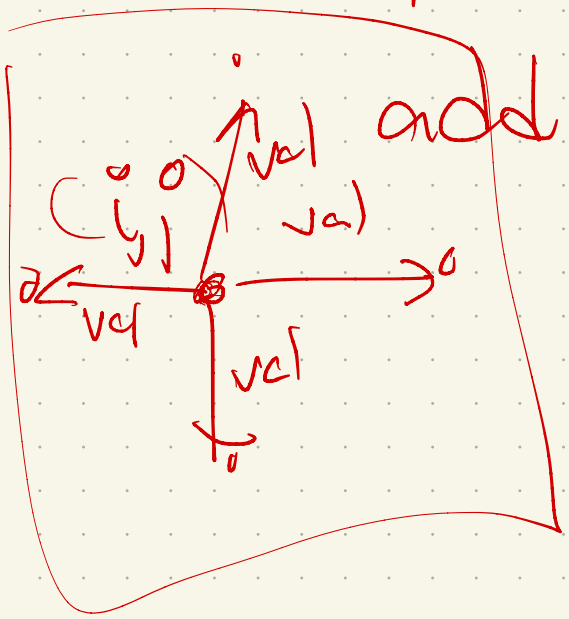
add at most 4 edges

let val = # in spot i,j

add edges:

- $i+val, j$ →
- $i-val, j$ ←
- $i, j+val$ ↑
- $i, j-val$ ↓

as long as nothing is $>n$ or <1.



So:

Graph: (V, E)

$V = n^2$ → (i, j) pairs

$E \leq 4 \cdot n^2$ → ≤ 4 moves per spot

use bool

WFS((vertex(1,1)) ← top left corner

↳ marked array: for each (i,j)

is there a path $(1,1) \rightarrow (i,j)$

check if (n,n) was marked

↳ return true

else false